

DOSYALAR VE AKIŞLAR

Dosyalar ve Klasörler

Kısaca **dosya** (**file**), verilerin bir araya geldiği (**collection of data**) ikincil saklama ortamıdır (**second storage**). Bu ikincil ortam; Bir bilgisayar **belleği** (**memory**) olabileceği gibi, verilerin elektrikler kesildiğinde kaybolmayacağı **sabit disk** (**hard disc**), ya da bir başka ortama/bilgisayara veri **gönderen** (**send**) veya **alan** (**receive**) modem ve benzeri bir **cihaz** (**device**) olabilir.

Dosyalar söz konusu olduğunda bazı kavramları bilmemiz gerekir;

- **Yol** (**Path**): Dosyanın dosya sistemindeki konumu. Örneğin; **C:\Users\ILHANOZKAN\Okubeni.txt**
- **Sonuna Ekleme** (**Append**): Dosya mevcutsa, dosyanın **sonuna veri ekler** (**append**) değilse yeni dosya üzerine yazar.
- **Metin** (**Text**): Yazılacak veya saklanacak verinin **metin** yani gözle okunabilir olduğunu ifade eder. Yani okunup yazılan veri basit **metin** editörleriyle görülebilir.
- **İçerik** (**Contents**): Yazılacak **dizgilerin** (**string**) bir koleksiyonu.
- **Kaynak** (**Source**): Kaynak dosyanın dosya sistemindeki konumu.
- **Hedef** (**Destination**): Hedef dosyanın dosya sistemindeki konumu

Veriler dosyalara aynı nehirde birbirinin peşi sıra giden tekneler gibi bayt-bayt gönderilir veya alınırlar. Bu nedenle dosyalara veri taşıyan ve veri getiren nesnelere **akış** (**stream**) adı verilir. Dosyaya giden nehre hangi veriyi koyarsak onu nehrin sonunda onu dosyaya koyar. Benzer şekilde nehirde hangi sırada veri gelirse o sırada verileri nehirde alırız.

Dosya Oluşturma

File statik sınıfının **Create** yöntemini kullanarak dosyalar oluşturabiliriz. Bu yöntem, verilen yolda dosyayı oluşturur, aynı zamanda dosyayı açar ve bize dosyanın **FileStream** nesnesini verir. İşimiz bittikten sonra dosyayı kapattığınızdan emin olmalıyız;

```
// See https://aka.ms/new-console-template for more information
FileStream dosyaAkışı = File.Create(@"\Metin.txt");
// Programın çalıştığı dizine dosya oluşturur.
dosyaAkışı.Close();
```

Buna eşdeğer olarak **Open** Yöntemini kullanabiliriz:

```
FileStream dosyaAkışı = File.Open(@"\Metin.txt", FileMode.Create);
// Programın çalıştığı dizine dosya oluşturur.
dosyaAkışı.Close();
```

using anahtar kelimesi ile açılan bir kaynağın kapatılmasını otomatik olarak yapabiliriz. Dosya yolundaki ters bölü karakteri metinler için özel karakter olduğundan iki kez yazılır;

```
using (var dosyaAkışı2 = File.Create("C:\\Users\\ILHANOZKAN\\YeniDosya.txt"))
{
    //dosyaAkışı kullanılmıyorsa blok dışında otomatik olarak kapatılır.
}
```

Tam metin (**verbatim string**) kullanarak ters bölü karakterini iki kez yazmaktan kurtulabiliriz;

```
File.Create(@"C:\Users\ILHANOZKAN\YeniDosya2.txt").Close();
/*
*/
```

Dosya oluşturma bir diğer yolu da **FileStream** sınıfını kullanmaktır;

```
FileStream dosyaAkışı3 = new FileStream(@"\Metin.txt",
                                         FileMode.OpenOrCreate,
                                         FileAccess.ReadWrite,
                                         FileShare.None);
```

FileMode: "Dosya oluşturulmalı mı? açılmalı mı? mevcut değilse oluşturulmalı mı, açılmalı mı?" türündeki soruları yanıtlar ²⁵.

FileAccess: "Dosyayı okuyabilmeli miyim, dosyaya yazabilmeli miyim yoksa her ikisini de yapabilmeli miyim?" türündeki soruları yanıtlar ²⁶.

FileShare: "Diğer kullanıcılar dosyayı aynı anda kullanırken okuyabilmeli, yazabilmeli mi vb.?" türündeki soruları yanıtlar ²⁷.

Dosya Taşıma

File statik sınıfının **Move** yöntemini kullanarak dosyaları taşıyabiliriz.

```
//File.Move("kaynakYol", @"hedefYol");
File.Move(@"C:\Users\ILHANQZKAN\YeniDosya2.txt", @"D:\YeniDosya.kopya.txt");
```

Yalnızca dosyanın dizinini değiştirir. Bu işlem dosya boyutuna göre zaman almaz. Ayrıca aynı dosya hedefte var ise kopyalama olmaz.

Dosya Silme

```
string path = @"D:\YeniDosya.kopya.txt";
File.Delete(path);
```

Delete yöntemi dosya mevcut değilse istisna oluşturmaz, ancak belirtilen yol geçersizse veya çağıran gerekli izinlere sahip değilse istisna oluşturur. **Delete** çağrılarını her zaman **try-catch** bloğunun içine koymalı ve tüm beklenen istisnaları işlemelisiniz. Birden fazla program aynı dosyayı silmeye kalkışabilir ve bu durumda **yarış durumu** (**race condition**) ortaya çıkar. Bunu önlemek için silme mantığını **lock** ifadesinin içine koymalıyız.

Bir Klasördeki Dosyaları Öğrenme

Klasördeki tüm dosyaları öğrenmek için aşağıdaki kodu yazarız;

```
var FileSearchRes = Directory.GetFiles(Path, "*.*", SearchOption.AllDirectories);
```

Belirtilen dizindeki tüm dosyaları (*.*) adı ve uzantısı ne olursa olsun demektir) temsil eden bir **FileInfo** dizisi döndürür. Eğer belirtilen dizin ve alt dizinlerindeki PDF uzantılı dosyaları arayacak isek aşağıdaki kodu yazmalıyız;

```
// See https://aka.ms/new-console-template for more information
string Path = @"D:\CSharp";
string[] FileSearchRes = Directory.GetFiles(Path, "*.pdf", SearchOption.AllDirectories);
foreach (var item in FileSearchRes)
{
    Console.WriteLine(item);
}
```

Metin Dosyasını Okuma

Bir dosyanın tüm içeriğini bir **dizgiye** (**string**) okumak için **System.IO.File.ReadAllText** ve yazmak için **System.IO.File.WriteAllText** fonksiyonunu kullanabilirsiniz;

```
// See https://aka.ms/new-console-template for more information
System.IO.File.WriteAllText(@".\Metin.txt",
    "Bu bir deneme metnidir.\nBu metin ikinci satırdır.");
string text = System.IO.File.ReadAllText(@".\Metin.txt");
string[] lines = text.Split(new[] { '\n' }, StringSplitOptions.None);
int lineCount = lines.Length;
Console.WriteLine($"Toplam {lineCount} satır var.");
```

²⁵ <https://learn.microsoft.com/en-us/dotnet/api/system.io.filemode?view=net-9.0&redirectedfrom=MSDN>

²⁶ <https://learn.microsoft.com/en-us/dotnet/api/system.io.fileaccess?view=net-9.0&redirectedfrom=MSDN>

²⁷ <https://learn.microsoft.com/en-us/dotnet/api/system.io.fileshare?view=net-9.0&redirectedfrom=MSDN>

Tüm satırları bir dizgi dizisine (`string[]`) okumak için `System.IO.File.ReadAllLines` ve yazmak için `System.IO.File.WriteAllLines` fonksiyonu kullanılır;

```
// See https://aka.stream/new-console-template for more information
string[] lines = System.IO.File.ReadAllLines(@"..\Metin.txt");
foreach (string line in lines)
{
    Console.WriteLine(line);
}
```

Büyük Metin Dosyaları

Büyük dosyalarla çalışırken, bir dosyadaki tüm satırları `IEnumerable<string>` jenerik ara yüzüne okumak için `System.IO.File.ReadLines` yöntemini kullanabilirsiniz;

```
// See https://aka.stream/new-console-template for more information
using System.Text; //Encoding için gerekli
IEnumerable<string> AllLines = File.ReadLines(@"..\Metin.txt", Encoding.Default);
foreach (string line in AllLines)
{
    Console.WriteLine(line);
}
```

Burada farklı [yerel kodlamaya](#) ([encoding](#)) sahip metin dosyasının okuma imkanına da sahip oluruz²⁸. Ayrıca `File.WriteLine` fonksiyonunu yazmak için de kullanabiliriz.

Akış Nedir?

Bir akış, veri aktarımı için düşük seviyeli bir araç sağlayan bir nesnedir. Kendileri veri kapsayıcıları olan koleksiyonlar gibi hareket etmezler. Akışlarda ele aldığımız veriler bayt dizisi (`byte []`) biçimindedir. Okuma ve yazma işlevlerini gerçekleştirecek akışlarda doğrudan tamsayılar, [dizgiler](#) (`string`) için hiçbir [yöntem](#) (`method`) yoktur. Bu da akışı genel amaçlı yapar, ancak yalnızca metin aktarmak istiyorsanız çalışması daha az basittir.

Akışlar, özellikle büyük miktarda veriyle uğraşırken çok yararlıdır. Hepsi n adet bayt üzerinden işlem yapar. Yazılması/okunması gereken yere (yani yedekleme deposuna) göre farklı türde [akış](#) (`stream`) kullanmamız gerekecek. Örneğin, kaynak bir dosyaysa, `FileStream` kullanmamız gerekir;

```
string filePath = @"..\Metin.txt";
using (FileStream fs = new FileStream(filePath,
                                     FileMode.Open,
                                     FileAccess.Read,
                                     FileShare.ReadWrite))
{
    // Dosya Akışı ile Yapılacak İşler
    fs.Close();
}
```

Benzer şekilde, yedekleme deposu bellek ise `MemoryStream` kullanılır;

```
// Diskte dosyada bulunan tüm baytları oku.
byte[] dosya = File.ReadAllBytes(@"..\Metin.txt");
// Okunan baytlar için hafızada bir akış oluştur.
using (MemoryStream memory = new MemoryStream(dosya))
{
    // Hafızadaki akış ile Yapılacak İşler
}
```

Benzer şekilde `System.Net.Sockets.NetworkStream` ise ağ işlemleri için kullanılır.

Tüm Akışlar, `System.IO.Stream` sınıfından türetilmiştir. Veriler doğrudan akışlardan okunamaz veya akışlara yazılamaz. .NET Framework içinde, temel veri tipler ve düşük seviyeli akış ara yüzü arasında

²⁸ <https://learn.microsoft.com/en-us/dotnet/api/system.text.encoding?view=net-9.0>

dönüşüm yapan ve verileri sizin için akışa veya akıştan size aktaran **StreamReader**, **StreamWriter**, **BinaryReader** ve **BinaryWriter** gibi yardımcı sınıflar bulunur.

Akışlara Veri Yazma ve Okuma

StreamWriter aracılığıyla akışları kullanırken ve bunları kapatırken dikkatli olmak gerekir.

```
// See https://aka.stream/new-console-template for more information
FileStream fs = new FileStream(@".\Metin.txt", FileMode.Create);
StreamWriter sw = new StreamWriter(fs);
string NextLine = "Bu metin dosyanın sonuna eklenir.";
sw.WriteLine(NextLine);
sw.Close();
//fs.Close(); Bu talimata gerek yoktur çünkü örtük olarak sw.Close() bu yöntemi çalıştırır.
```

Benzer şekilde **StreamReader** kullanırken aynı şeye dikkat etmemiz gerekir;

```
// See https://aka.stream/new-console-template for more information
using (var stream = new MemoryStream())
{
    StreamWriter writer = new StreamWriter(stream);
    writer.WriteLine(123);
    writer.WriteLine("Bu bir deneme metnidir.");
    //writer.Close(); Bu akışı kapatır.
    writer.Flush(); //Kalan verileri akışa gönderebilirsiniz.
    //Kapatma yapılırsa bu otomatik olarak yapacaktır
    stream.Position = 0; // Akışın başına geri dön
    StreamReader sr = new StreamReader(stream);
    var myStr = sr.ReadToEnd();
    Console.WriteLine(myStr);
    /*
    123
    Bu bir deneme metnidir.
    */
    sr.Close();
    stream.Close();
}
```

Akış Olmadan Metin Dosyaları Oluşturma

StreamWriter nesnesinin kapsam dışına çıktığı anda elden çıkarılmasını ve böylece dosyanın kapatılmasını sağlayan **using** anahtar sözcüğünün kullanılması gerekir.

```
// See https://aka.ms/new-console-template for more information
string[] lines = { "Birinci Metin Satırı.", "İkinci Metin Satırı.", "Üçüncü Metin Satırı" };
using (System.IO.StreamWriter sw = new System.IO.StreamWriter(@"D:\Metin.txt"))
{
    foreach (string line in lines)
    {
        sw.WriteLine(line);
    }
} //<-Bu bloktan sonra sw çöpe gönderilir.
```

StreamWriter sınıfı yapıcısında ikinci bir bool parametresi alabileceğini ve bu sayede dosyaya ekleme yapılabilir.

```
bool appendExistingFile = true;
using (System.IO.StreamWriter sw2 = new
System.IO.StreamWriter(@"D:\Metin.txt", appendExistingFile))
{
    sw2.WriteLine("Bu satır son satır olarak dosya sonuna eklenecek.");
}
```

İkili Dosyaya Yazma ve Okuma

Metin dosyalarından farklı olarak ikili olarak sayı ve yapıları dosyalara yazabilir ve okuyabiliriz. Bu şekilde oluşan dosyalar gözle okunamazlar. Çünkü işlemcinin doğal olarak anladığı ikili sayı sisteminde veriler dosyalara yazılır ve okunur.

```
// See https://aka.stream/new-console-template for more information
using System.Text;
string fileName = "ikili.bin";
using (var stream = File.Open(fileName, FileMode.Create))
{
    using (var writer = new BinaryWriter(stream, Encoding.UTF8, false))
    {
        writer.Write(1.250F);
        writer.Write(@"c:\Temp");
        writer.Write("İlhan Özkan");
        writer.Write(10);
        writer.Write(true);
    }
}

float aspectRatio;
string tempDirectory;
string tempUserName;
int autoSaveTime;
bool showStatusBar;
if (File.Exists(fileName))
{
    using (var stream = File.Open(fileName, FileMode.Open))
    {
        using (var reader = new BinaryReader(stream, Encoding.UTF8, false))
        {
            aspectRatio = reader.ReadSingle();
            tempDirectory = reader.ReadString();
            tempUserName = reader.ReadString();
            autoSaveTime = reader.ReadInt32();
            showStatusBar = reader.ReadBoolean();
        }
    }
    Console.WriteLine("Aspect ratio set to: " + aspectRatio);
    Console.WriteLine("Temp directory is: " + tempDirectory);
    Console.WriteLine("User Name is: " + tempUserName);
    Console.WriteLine("Auto save time set to: " + autoSaveTime);
    Console.WriteLine("Show status bar: " + showStatusBar);
}
```

Oluşan **ikili.bin** dosyasına ilk önce **kayan noktalı sayı** (**float**) değeri sonra iki adet **dizgi** (**string**) ardından bir **tamsayı** (**int**) ve **bool** değer ikili olarak yazılmıştır. Bellekte saklandığı şekliyle dosyaya kaydedildiğinden içeriği gözle okunamaz. İkili dosyaları açan görüntüleyiciler ile açılıp görülebilir. Bir ikili dosyayı onaltılık rakamlara çevirip metin olarak görüntülemek için Windows ortamında aşağıdaki komut verilebilir;

```
C:\Users\ILHANOZKAN>certutil -encodehex ikili.bin ikili.txt
Input Length = 31
Output Length = 147
CertUtil: -encodehex command completed successfully.
```

Artık dönüştürülen dosyayı (**ikili.txt**) istediğimiz metin düzenleyicisinde (**notepad**) açıp okuyabiliriz.

```
0000  00 00 a0 3f 07 63 3a 5c  54 65 6d 70 0d c4 b0 6c  ...?.c:\Temp...l
0010  68 61 6e 20 c3 96 7a 6b  61 6e 0a 00 00 00 01  han ..zkan.....
```